

PRÁCTICA EVALUABLE

Tienda Deportiva Avanzada

Objetivo:

Desarrollar una aplicación en Java para gestionar una tienda de deportes, implementando conceptos de herencia, interfaces y manejo de excepciones para gestionar productos y compras.

Requisitos Funcionales:

1. Gestión de Productos:

- La tienda debe manejar distintos tipos de productos (por ejemplo, Equipamiento y Ropa Deportiva).
- Cada producto tiene un código único, nombre, precio y stock.

2. Operaciones sobre Productos:

- **Agregar producto:** Permitir añadir nuevos productos al inventario.
- **Eliminar producto:** Permitir eliminar un producto usando su código.
- **Actualizar producto:** Permitir actualizar el precio y/o stock de un producto existente.

3. Gestión de Compras:

- Realizar una compra reduciendo el stock del producto correspondiente.
- Si se intenta comprar una cantidad mayor a la disponible, se debe lanzar una excepción personalizada (por ejemplo, `StockInsuficienteException`).
- Registrar cada compra con fecha, producto, cantidad y precio total.

4. Interfaz de Usuario:

- La aplicación debe presentar un menú interactivo con opciones para:
 1. Mostrar inventario.
 2. Agregar producto.
 3. Eliminar producto.
 4. Actualizar producto.
 5. Realizar compra.
 6. Mostrar historial de compras.
 7. Salir.
-

Diseño de Clases e Interfaces:

1. Interfaz Producto:

- **Métodos a implementar:**
 - `String getCodigo()`
 - `String getNombre()`

- `double getPrecio()`
- `int getStock()`
- `void descontarStock(int cantidad)`
- `String getInfo()` // Devuelve información detallada

2. Clase Abstracta **ProductoBase** (opcional, para evitar duplicidad):

- **Atributos comunes:**
 - `codigo`, `nombre`, `precio` y `stock`
- **Implementa la interfaz **Producto**:**
 - Provee métodos comunes como getters, setters y `descontarStock()`.
- **Método abstracto:**
 - `String getInfo()`

3. Subclases de **Producto**:

- **Equipamiento:**
 - **Atributos adicionales:**
 - Por ejemplo, `String deporte` o `String marca`.
 - **Implementa `getInfo()`:**
 - Debe mostrar los datos generales junto con los específicos.
- **Ropa Deportiva:**
 - **Atributos adicionales:**
 - Por ejemplo, `String talla` y `String material`.
 - **Implementa `getInfo()`:**
 - Mostrar todos los detalles del producto.

4. Clase **Compra**:

- **Atributos:**
 - `LocalDate fecha` (o una cadena con la fecha)
 - `Producto producto`
 - `int cantidad`
 - `double precioTotal`
- **Métodos:**
 - Constructor, getters y `toString()` para mostrar la información de la compra.

5. Clase **Tienda**:

- **Atributos:**
 - `List<Producto> inventario`
 - `List<Compra> historialCompras`
- **Métodos principales:**
 - `void añadirProducto(Producto p)`
 - `void eliminarProducto(String codigo)`
 - `void actualizarProducto(String codigo, double nuevoPrecio, int nuevoStock)`
 - `void realizarCompra(String codigo, int cantidad)`
 - Buscar el producto *por código*.

- Verificar que el stock sea suficiente; si no, lanzar `StockInsuficienteException`.
- Actualizar el stock y registrar la compra.
- `void mostrarInventario()`
- `void mostrarHistorialCompras()`

6. **Excepción Personalizada `StockInsuficienteException`:**

- Es una clase que **extiende** de `Exception`.
- Se lanza cuando se intenta comprar más unidades de las disponibles.
- Debe mostrar un mensaje descriptivo (por ejemplo, "Stock insuficiente para el producto [código]").

7. **Clase `Aplicacion` (con el método `main`):**

- Instanciar la tienda y precargar algunos productos.
- Presentar un menú interactivo al usuario para realizar las operaciones.
- Utilizar bloques `try-catch` para capturar y manejar la `StockInsuficienteException` y otras entradas erróneas.

Pistas y Consejos:

- **Organización:**

Crea un nuevo proyecto para el ejercicio.

Puedes organizar tu código en paquetes (por ejemplo, `modelo`, `excepciones`, `aplicacion`) para mantener una estructura limpia.

- **Validaciones:**

Asegúrate de validar la entrada del usuario en el menú, evitando que se ingresen valores no numéricos donde se requiera un número.

- **Herencia e Interfaces:**

La creación de una clase abstracta (`ProductoBase`) que implemente la interfaz `Producto` te ayudará a reducir duplicación de código en las subclases.

- **Manejo de Fechas:**

Para la fecha de la compra, puedes usar `LocalDate` y formatearla según se necesite.

- **Manejo de Excepciones:**

Implementa la excepción `StockInsuficienteException` para que, en el método `realizarCompra`, si el stock no es suficiente, se lance y se capture en el `main` mostrando un mensaje adecuado.

